Hospital Efficiency and Patient Outcome Analytics

A Healthcare Analytics Pipeline Built on a Synthetic MIMIC-IV-Style Dataset

Bryan Marin Samano
October 25, 2026

Table of Contents

Hospital Efficiency and Patient Outcome Analytics 1

Abstract 3

1. Introduction 4

2. Dataset and Data Preparation 5

3. Methodology 6

4. Reproducibility 7

5. Operational Results 9

6. Patient Outcome and Predictive Results 10

7. Discussion 12

8. Limitations and Future Work 14

9. Conclusion 15

References 17

Abstract

Hospitals have extremely granular data on every admission (lab orders, ICU transfers etc.) and that data can tell you where the resources are going and which patients are at the greatest risk. This project builds a full analytics pipeline on a synthetic dataset which is based on the relational schema of MIMIC-IV. It covers the full range of the pipeline from data cleaning through to feature engineering, SQL, descriptive stats and machine learning. It's structured around two questions: how efficiently is the hospital running and which patient characteristics are associated with poor outcomes?

Of 25,059 admissions: mean length of stay 10.0 days, mean ICU length of stay 4.0 days, 30-day readmission rate 14.9%, hospital mortality 10.0%. The differences between the admission types were very small (no difference in LOS or readmission rate exceeded two tenths of a days or one percentage point respectively). For predicting 90-day mortality (8.4% of admissions) logistic regression (ROC-AUC 0.660, recall 66.9%) performed better than random forest (ROC-AUC 0.604, recall 46.9%) although both models had very low precision. Hospital length of stay was the only feature that was meaningfully associated with 90-day mortality and there were several features in the data which suggested the outcomes were being generated with little or no relationship to actual clinical severity. The results therefore describe the operation of the pipeline rather than the performance of any real hospital. Even though the data are synthetic, the findings are important because they show a pipeline that could be used in real hospitals. The pipeline’s ability to reproduce expected clinical patterns, even subtle ones, supports its validity and suggests it could help provide useful analytics with real data.

1. Introduction

Running a hospital involves the concurrent management of beds, staff, ICU capacity, medications and diagnostic services, each of which produces data. You can use that data to work out where patients spend their time, what services are used most frequently and who is most likely to be readmitted or to die. Machine learning has been applied to many of these problems using electronic health records (Esteva et al. 2019) but a model that performs well in terms of prediction is not necessarily a useful model in practice. Rajkomar et al. (2019) make the point that clinical models need to be evaluated not just on their predictive performance but on how well they work in the context of care and what the cost of their errors would be.

This project goes through the full process on a synthetic dataset which is designed to mimic MIMIC-IV, thus enabling the methods to be developed and shared openly without access to actual patient records. Synthetic data is used rather than real patient records to allow for open development and sharing without any privacy issues. The caveat with this approach is that the results will not be fully generalizable and the patterns we see here may not be representative of the real-world clinical population. This project poses two questions:

1. How efficiently is the hospital operating, and where are the possible pressure points?
2. Which patient characteristics are associated with poor outcomes?

The first question is addressed via length of stay, ICU use, a resource use score and 30-day readmission rate, all compared across admission type. The second is answered with some descriptive statistics and two predictive models, logistic regression and random forest, which are trained to predict 90-day mortality. This is not about building a clinical tool. It's about demonstrating how raw relational data can be transformed into operational metrics and predictive outcomes and what those outcomes can and can’t tell us.

2. Dataset and Data Preparation

MIMIC-IV is a de-identified database of hospital and ICU admissions from Beth Israel Deaconess Medical Centre, stored as linked tables (patient, admission, ICU stay, diagnoses, and other clinical events) (Johnson et al. 2023, 2024). The dataset used here isn't MIMIC-IV, but a synthetic dataset which replicates the general table structure of MIMIC-IV, with 10,200 patients and 25,059 admissions across the nine files as listed in Table 1.

Table 1. *Source tables in the synthetic dataset*

Table	Contents	Rows
patients.csv	Age, sex, and race for each patient	10,200
admissions.csv	Admission and discharge dates, LOS, admission type, discharge location	25,059
icu_stays.csv	ICU length of stay (one row per admission)	25,059
diagnoses.csv	Diagnoses for each admission	74,810
procedures.csv	Procedures for each admission	62,542
medications.csv	Medications for each admission	100,576
labs.csv	Laboratory tests and values	363,744
comorbidities.csv	Chronic conditions for each patient	19,961
outcomes.csv	SOFA score, mortality, 90-day mortality, readmission, sepsis, ventilation	25,059

Cleaning began with the removal of exact duplicates (this removed 200 duplicate patient records and 1,437 duplicate lab records). Missing race values were coded as "UNKNOWN" and inconsistent sex values (e.g. "Male" and "M") were standardized. Lab values stored as text were converted to numbers and any lab value which was missing, or which could not be converted was replaced by the median value for that test.

As a single admission can have multiple diagnoses, multiple medications, multiple procedures and multiple labs results the raw tables would have been joined directly, and this would have led to a massive multiplication of rows. To avoid this, each event table was grouped down to one row per admission (e.g. total number of lab records, total number of medications, total number of procedures) and then these were joined onto the admissions table. The patient demographics, ICU and outcomes tables already had one row per patient or admission, so these were joined directly onto the admissions table. The

final row count was 25,059 post all the joins which confirms that no admissions were duplicated. The cleaning and joining were all done in pandas (McKinney, 2022) using NumPy (Harris et al., 2020).

After the merge any remaining missing values were filled in with zero. These were almost all from the count and comorbidity columns (an admission with no corresponding rows in procedures.csv, or a patient with no corresponding rows in comorbidities.csv will have no value after a left join), so for those columns zero is the appropriate value. In real life records the same approach would be far more dangerous as a missing lab value could mean the test was never ordered, the result was in another system, or the documentation was simply incomplete.

I also created a resource score which was simply the number of lab records, medications and procedures for that admission. It's a fairly straightforward way of measuring the level of clinical activity in an admission, though it treats a routine blood and a major operation as being the same in terms of resource usage, so it isn't an appropriate measure of cost or staff time at all.

3. Methodology

All the analysis was done in Python in Google Colab. DuckDB (DuckDB, n.d.) was used to run SQL directly against the cleaned master dataset which meant that it was easy to calculate hospital wide and admission type indicators via GROUP BY queries rather than a long series of pandas operations. The figures were generated using Matplotlib (Hunter, 2007) and the models were built using scikit-learn (scikit-learn developers, n.d.).

For the operational question I calculated the average and median length of stay (LOS), average ICU LOS, average resource score, 30-day readmission and hospital mortality. I did this for the whole hospital and then for each admission type (observation, emergency, urgent and elective). An admission type which stood out on several of the metrics at once would be a potential pressure point. None of the metrics I've used measure efficiency directly. For example, LOS will be affected by the sensitivity of the patient and whether a rehab bed is available at discharge.

For the outcome (death within 90 days) nine numeric predictors were used: age, SOFA score, hospital LOS, ICU LOS, resource score, a comorbidity score and the counts of diagnoses, medications and lab records. These were selected based on the idea that they represent a range of different aspects of patient status and hospital utilization. Age is a demographic variable, the comorbidity flags are a reflection of the patient's comorbid state, the SOFA score is a measure of the severity of illness, the hospital and ICU LOS are measures of length of stay and the resource score together with the counts of the various clinical events (diagnoses, meds, labs) is a measure of the pattern of utilization. The aim was to build a model that would capture a range of potential determinants of 90-day mortality in the synthetic data. For reproducibility all the key model hyperparameters (e.g. the logistic regression penalty, the number of trees in the random forest, the maximum depth) and allalle feature engineering steps are described in detail in the following section. Logistic regression was used as an interpretable baseline (Hosmer et al., 2013) and the random forest was included as it can capture non-linear effects and interactions without requiring these to be specified in advance (Breiman, 2001). The data was split 80/20 into a training set (20,047 admissions) and a test set (5,012 admissions).

Only 8.4% of the admissions resulted in death within 90 days so that measure of accuracy would be highly misleading. A model which predicted survival for all patients would be correct 92% of the time. The models were therefore assessed on precision, recall, F1 and ROC-AUC with recall being given extra weight as in a screening context it's generally far worse to miss a patient who will die than to flag a patient who won't. To understand the models, the logistic regression coefficients, the random forest impurity-based feature importance and the simple correlation of each feature with 90-day mortality were considered.

4. Reproducibility

To make the analysis fully reproducible every model setting and feature engineering step required to re-run it was reported. The analysis was carried out in Python 3.13 using pandas 2.2.3, NumPy 2.1.3, scikit-learn 1.6.1 and DuckDB 1.3.2. A random seed of 42 was used for both the train/test split and for the

random forest so that a re-run of the code would produce the same split and the same results. The split was stratified by the outcome to ensure that the proportion of deaths is the same in the training and test sets (420 of the 5,012 test admissions).

Each row in the modelling dataset is one admission. Age is taken from the anchor_age field in patients.csv so a patient has a single age value for all their admissions rather than an age being calculated for each visit. Hospital length of stay is from admissions.csv (in whole days) and ICU length of stay is from icu_stays.csv (one ICU stay per admission). Diagnoses, medications, procedures and lab records are all counted per admission and the resource score is the sum of the last three. The SOFA score, 90-day mortality and 30-day readmission are all taken from outcomes.csv with one value per admission.

There are two features that are needed to explain in detail. Firstly, the column 'charlson_index' is not the actual Charlson Comorbidity Index. It is a derived score which is simply 0.8 x the number of diagnoses, so it contains the same information as the number of diagnoses. Secondly yes/no flags were created for the 5 chronic conditions (CHF, CKD, COPD, diabetes and hypertension) and the mean, minimum, maximum and standard deviation of each of the lab values for each admission. These were used in the descriptive analysis only and were not used as predictors in the models (neither were sex, race or admission type so no need for categorical encoding). All 9 predictors were standardized using StandardScaler, fitted on the training set only, prior to training both models. Standardization is important for logistic regression and has no effect on a random forest. The model settings are listed in Table 2.

Table 2. *Model hyperparameters and settings*

Model	Setting	Value
Logistic Regression	Penalty	L2 (default)
Logistic Regression	Regularization strength (C)	1.0 (default)
Logistic Regression	Solver	lbfgs (default)
Logistic Regression	Maximum iterations	5,000
Logistic Regression	Class weighting	Balanced
Logistic Regression	Classification threshold	0.50 (default)
Random Forest	Number of trees	300
Random Forest	Maximum depth	Unlimited (default)
Random Forest	Minimum samples per leaf	1 (default)

Model	Setting	Value
Random Forest	Features considered per split	Square root of total (default)
Random Forest	Class weighting	Balanced
Random Forest	Classification threshold	0.10
Random Forest	Random seed	42

Balanced class weighting means that each death is worth roughly 11 times as much as each survival during training, so the models don't just learn to predict survival for everyone. No hyperparameter tuning was done, and all the unlisted settings were left at their scikit-learn defaults. The random forest threshold was reduced from 0.50 to 0.10 as I'd played around with a few different thresholds on the training data and picked the one that gave the best balance of recall and precision.

5. Operational Results

Table 3 shows the hospital wide figures. Mean and median length of stay were both 10 days, and the distribution was almost perfectly flat (stays ranged from 1 to 19 days, quartiles at 5, 10 and 15). Real hospital length of stay is normally right skewed (most patients will have left in a few days; a small number will have stayed for weeks). A flat distribution like this is an early warning sign that length of stay has been generated at random within a fixed range.

Table 3. *Hospital-wide operational measures*

Measure	Result
Average hospital LOS	≈ 10.00 days
Median hospital LOS	≈ 10.00 days
Average ICU LOS	≈ 3.99 days
Average resource score	≈ 20.37
30-day readmission rate	≈ 14.86%
Hospital mortality	≈ 9.96%
90-day mortality	≈ 8.37%

Table 4. *Operational measures and hospital mortality by admission type*

Admission Type	Admissions	Avg. LOS (days)	Avg. ICU LOS (days)	Avg. Resource Score	30-Day Readmission	Hospital Mortality
Observation	6,236	≈ 10.07	≈ 4.01	≈ 20.43	≈ 14.50%	≈ 10.20%
Emergency	6,281	≈ 9.89	≈ 3.99	≈ 20.43	≈ 15.17%	≈ 9.97%
Urgent	6,210	≈ 10.06	≈ 3.99	≈ 20.33	≈ 15.36%	≈ 9.98%
Elective	6,332	≈ 9.99	≈ 3.99	≈ 20.28	≈ 14.42%	≈ 9.71%

Observation admissions had the highest average LOS, ICU LOS, resource score and hospital mortality (Table 4). Urgent admissions had the highest readmission rate, very close to that of emergency admissions. Elective admissions had the lowest readmission rate, mortality and resource score which is as you would expect with the most severe patients being admitted on an emergency or urgent basis.

The key finding is just how small the differences are. The difference in average length of stay between the longest and shortest admission type is less than a fifth of a day. The readmission rates are all within one percentage point of each other and the four admission types are virtually identical in size. In a real hospital you'd expect emergency and elective patients to be vastly different from each other and observation stays to be considerably shorter than the other types (observation is for patients you expect to discharge in a day or two). The most likely explanation is that the synthetic data generator simply assigned admission type at random, independently of the clinical variables. Differences this small shouldn't be interpreted as real pressures. The analysis does confirm that the SQL pipeline produces the correct breakdowns and the same queries would be informative on real data where the admission types are different.

6. Patient Outcome and Predictive Results

Prior to the modelling, it was inspected how 90-day mortality varied with each individual patient characteristic and the answer was essentially nothing. It ranged from 8.1% to 8.7% across the age groups, didn't increase with SOFA score and didn't increase with the comorbidity score. Of the nine model features only hospital length of stay had any real correlation with 90-day mortality (r=0.17). All the other features had an absolute correlation below 0.02 (Table 6). This is important for everything that follows as the model cannot find a relationship that isn't there in the data.

Table 5. *Model performance for 90-day mortality on the test set*

Metric	Logistic Regression	Random Forest
Classification threshold	0.50	0.10
Precision	13.05%	11.16%
Recall	66.90%	46.90%

Metric	Logistic Regression	Random Forest
F1 score	21.83%	18.02%
ROC-AUC	0.660	0.604
Deaths correctly flagged	281 of 420	197 of 420
False positives	1,873	1,569

Logistic regression picked up 281 of the 420 deaths in the test set (recall = 66.9%) as opposed to 197 for the random forest (Table 5). The precision was very low for both models. The logistic regression model flagged 2,154 admissions and of those only 13% resulted in death. Given that the base rate is 8.4% that's only about 1.6x better than just flagging admissions at random and in reality, it would be about seven false alarms for every true case.

The two models were run at different thresholds, so the precision and recall aren't a direct like-for-like comparison. ROC-AUC is threshold independent and, in that metric, logistic regression is again clearly superior (0.660 v 0.604), so the overall ranking is unchanged.

Why the simpler model wins is obvious from the correlations. Almost all the signal comes from one feature (hospital length of stay) and the relationship with mortality is very nearly linear which is precisely what logistic regression is designed to capture. The standardized coefficient for LOS is 0.71 which is four times greater than that of any of the other features. The random forest on the other hand was allowed to grow 300 trees with no depth limit. With 8 of its 9 features essentially carrying no signal those deep trees end up fitting the noise in the training data and that in turn degrades its performance on the test set. A depth limit on the trees or a minimum number of samples per leaf would probably close the gap.

Table 6. *Feature correlations with 90-day mortality and model importance*

Feature	Correlation with 90-day mortality	Logistic regression coefficient (standardized)	Random forest importance
ICU LOS	0.016	0.047	0.192
Hospital LOS	0.166	0.707	0.182
Age	0.005	0.003	0.161
SOFA score	0.008	0.017	0.114
Resource score	0.005	−0.175	0.107
Lab count	0.005	0.177	0.098
Medication count	0.005	0.055	0.071
Simplified comorbidity score	−0.001	−0.005	0.037

Feature	Correlation with 90-day mortality	Logistic regression coefficient (standardized)	Random forest importance
Diagnosis count	−0.001	−0.005	0.037

Table 6 also illustrates why the random forest feature importance is so misleading. The top three features were ICU length of stay, hospital length of stay and age. Both ICU LOS and age have very little to do with mortality. Impurity based importance tends to favor continuous features with lots of distinct values as they provide more potential split points. ICU LOS is recorded to 2 decimal places, and age has a lot of distinct values whereas the count features have very few. The importance ranking is therefore just a reflection of how often the feature was used to split (a lot of that will be on noise) rather than how useful it actually is for prediction. Permutation importance (which measures the drop in performance when a feature is shuffled) would give a much more honest ranking.

The logistic regression coefficients have their own oddity. The lab count (0.18) and the resource score (-0.18) have very similar in magnitude and opposite in sign coefficients. This is due to the resource score already incorporating the lab count so there is overlap between the two features and the model effectively splits the effect of the two between them to the extent that they cancel each other out. The same overlap is what causes the diagnosis count and the simplified comorbidity score to have identical coefficients (one is just a rescaled version of the other). Again, this doesn't imply that either feature is protective or harmful. A cleaner model would simply contain one of the features from each of these overlapping pairs.

Finally, the one strong predictor has a practical issue. Hospital length of stay isn't known until discharge so a model which used it wouldn't be able to flag high risk patients at admission (which is precisely when you'd want to be doing it). A model designed for early identification would have to exclude LOS and be based on information available on day one. In that dataset that would leave virtually no signal.

7. Discussion

The two halves of the project overlapped. Hospital length of stay was the primary operational metric and the only actual predictor of 90-day mortality. In real life you'd expect that link to exist (sicker

patients both have longer stays and are more likely to die). It also means that a long average stay in one unit isn't necessarily a sign of inefficiency, but it may simply reflect the patients that unity treats. In this case however SOFA score, age and comorbidities were all unrelated to mortality, so LOS isn't proxying for severity. The link was almost certainly baked directly into the data generator. A useful reminder that a model can only learn the patterns that are present in the data it's trained on and with synthetic data those patterns are precisely those that the generator has decided to include.

Finding the actual bottlenecks would also require data that isn't in this dataset at all (bed occupancy over time, ED wait times, discharge delays, staffing levels etc.). Those are the measures that tell you where the patient is actually waiting rather than just the total length of stay.

On the modelling side the results are a good illustration of why one metric is not sufficient. The logistic regression has the best ROC-AUC and best recall, but a clinician getting alerts from that model would be left with the impression that most patients flagged were not actually at high risk. Class weighting also causes the predicted probabilities to be pushed up so the outputs from neither model can be interpreted as the patient's true risk of death. Before any real use of such a model you'd need calibration, a decision threshold based on the cost of false positives vs false negatives, performance across different demographic groups and validation on out of sample data (Rajkomar et al., 2019). As Topol (2019) points out such predictions should be used to assist clinical judgement rather than to replace it.

The primary value of the project is the pipeline. It involved getting the one-to-many joins right, working out how to deal with missing data, feature engineering, writing the SQL for the operational KPIs and then evaluating the models on the imbalanced outcome. Equally important in the process of building it was the act of looking at the results and seeing where the data didn't behave like real clinical data. All of that would be directly transferable to a real dataset. The most difficult part of the project was the integration of the various healthcare tables, cleaning the data and then creating the features for both the operational analysis and the mortality prediction. One of the more surprising results was that Logistic Regression beat Random Forest on all four of the metrics reported. It was a clear demonstration that a more complex model isn't necessarily better.

8. Limitations and Future Work

The major constraint is that the data are synthetic and several of the checks indicate that the generator produced each variable essentially independently. LOS is uniformly distributed, the admission types are all equal size and have very similar outcomes, and the severity of illness measures (SOFA) show no correlation with death. The outcome labels are also internally inconsistent. Of the 2,497 admissions which are classified as in-hospital deaths, only 270 are classified as deaths within 90 days (which would be expected if you're dead in hospital, you're also dead at 90 days). There are also several admissions which have a discharge destination of ‘expired’ which are also classified as survivors. Not all of the findings can be deduced to real patients or real hospitals.

Other limitations were identified previously: the resource score is unvalidated, the comorbidity score is simply the count of diagnoses rescaled rather than a true Charlson comorbidity score, LOS is a raw proxy for efficiency and several of the model features are redundant. The models were evaluated on a single 80/20 split with no cross-validation or model tuning so the results would be different with a different split. Privacy wasn't an issue with the synthetic data but with the real MIMIC-IV data via the PhysioNet database, you do need to go through the full human subjects training process and have a signed data use agreement (Johnson et al. 2024).

The most useful next step would be to run the same pipeline on actual MIMIC-IV data. Then calculate a proper Charlson index from the diagnosis codes, add in the comorbidity flags and the lab summaries as predictors, get rid of the redundant features and build a version of the mortality model that's based solely on information available at admission. On the modelling side a stratified cross validation would be needed, tuning the depth and leaf size of the random forest, trying out gradient boosting and replacing the impurity-based importance with permutation importance or SHAP values would also be done. For the operational question one would use the time-stamped data on transfers and discharges to do a proper patient flow analysis rather than just work with averages.

9. Conclusion

This was a full healthcare analytics pipeline (synthetic data generation through to SQL analysis and ML) on a dataset derived from MIMIC-IV. The hospital in question had a 10-day average length of stay and a 14.9% readmission rate with very little difference between the various admission types (again, this is likely a reflection of the data generation process rather than any real differences in the patient population). For 90-day mortality the logistic regression model performed better than the random forest, though both models were extremely low in precision and hence the difference between a model that runs and one that is clinically useful is very clear. The best predictors were ICU and hospital length of stay (both of which are measures of how unwell the patient was) and neither of those would be known at the point in the admission where any intervention could be made.

These results point to two wider lessons. Firstly, synthetic data is of value in the construction and testing of a pipeline, but it is of no use in enabling one to draw a clear conclusion from it. The uniform admission type results are a case in point in that the synthetic dataset will only reproduce the relationships that were built into it and so the findings from it will describe the generator as much as the hospital. Secondly, operational and outcome analytics are more informative when viewed together than when viewed in isolation. The SQL summaries told you where the resources were going, and the models told you which patients were at risk and reading the two together provided a much more complete picture than either one by itself.

There are several steps that would take this to a point where it could be used in practice. The models would need retraining on real MIMIC-IV data and restricted to features that are available at or very near admission so that the predictions are coming in early enough to be of use. The decision thresholds would need to be adjusted to improve precision, and calibration would need to be checked to ensure that the predicted risks match the observed risks. Any deployment would also of course require data governance, privacy protections and input from the clinicians who would be the ones ultimately deciding how the predictions are used at the bedside. With those changes the same pipeline could answer the research questions with a greater certainty.

The synthetic dataset and all code used for data processing, analysis, and figure generation are available at: [GITHUB_REPOSITORY_LINK]. Supplementary materials, including extended methods, tables, and figures, are available in the same repository.

References

Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32. https://doi.org/10.1023/A:1010933404324

DuckDB. (n.d.). DuckDB documentation (version 2.0). https://duckdb.org/docs/2.0/

Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., Cui, C., Corrado, G., Thrun, S., & Dean, J. (2019). A guide to deep learning in healthcare. *Nature Medicine*, 25, 24–29.https://doi.org/10.1038/s41591-018-0316-z

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.https://doi.org/10.1038/s41586-020-2649-2

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression* (3rd ed.). https://dl.icdst.org/pdfs/files4/7751d268eb7358d3ca5bd88968d9227a.pdf

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.https://doi.org/10.1109/MCSE.2007.55

Johnson, A. E. W., Bulgarelli, L., Shen, L., et al. (2023). MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data*, 10.https://doi.org/10.1038/s41597-022-01899-x

Johnson, A., Bulgarelli, L., Pollard, T., Gow, B., Moody, B., Horng, S., Celi, L. A., & Mark, R. (2024). MIMIC-IV (version 3.1). *PhysioNet*. RRID:SCR_007345.https://doi.org/10.13026/kpb9-mt58

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). https://www.lkhibra.ma/books/Python-for-Data-Analysis.pdf

Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. The New England *Journal of Medicine*, 380, 1347–1358.https://doi.org/10.1056/NEJMra1814259

scikit-learn developers. (n.d.). Scikit-learn: Machine learning in Python. https://scikit-learn.org/stable/

Topol, E. J. (2019). *Deep medicine: How artificial intelligence can make healthcare human again*(1st ed.). Basic Books.

Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2